

5. Reverse Engineering



What you need today

◆ Kali VM

- ◆ Install gdb-peda as per lab 3 instructions

◆ Windows VM

- ◆ Download and install the OllyDBG debugger
 - ◆ <https://www.ollydbg.de/download.htm>

◆ Things we do today

- ◆ Reverse engineering:
 - ◆ Codes
 - ◆ x86 programs

Reverse Engineering

◆ What is Reversing?

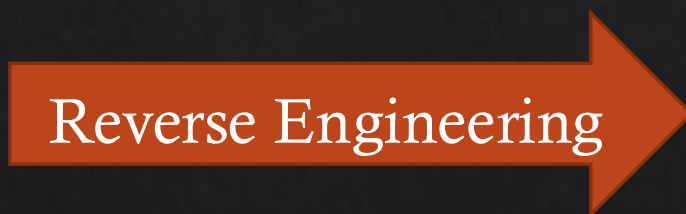
◆ What is Reverse Engineering?

Program Abstractions

1. Computers understand binary code
2. Binary code can be written in hexadecimal
3. Hexadecimal code can be encoded in assembly language
4. Assembly language is human-readable but not as intuitive as source code
5. Decompilers convert assembly into an easier-to-read source code



11001111 10101 == CD21 == int 21



Security-related Reversing

◆ Malicious software

- ◆ Malicious software (e.g., viruses and worms) spread much faster now that computers are connected to the Internet.
- ◆ The infection process used to be slow (e.g., viruses spread by diskette sharing).
- ◆ Today's worms can spread automatically to computers without any human intervention.
- ◆ Developers of malware use RE to find vulnerabilities in OS and application software.
- ◆ Developers of ant-virus software use RE to develop tools to protect users from malware.

◆ Reversing cryptographic algorithms

- ◆ Restricted algorithms, where the encryption is in the algorithm, (e.g., Caesar cipher) can be broken easily using RE.
- ◆ Key-based algorithms, where the algorithms are public but the key is secret, are more difficult to break.
 - ◆ Vulnerabilities include obtaining the key (need luck)
 - ◆ trying all possible combinations until you get the key (need quantum computer)
 - ◆ Look for a flaw in the algorithm that exposes the key of the original message (need RE)

Security-related Reversing

◆ Digital Right Management (DRM)

- ◆ Media content providers have developed or use technologies (DRM) that control the distribution of digital content (e.g., music, movies).
- ◆ DRM technologies determine if the content should be made available or not.
- ◆ Crackers use RE to attempt to defeat DRM technologies. RE can help to:
 - ◆ reveal the secrets of DRM technology;
 - ◆ discover simple modifications that can be made to DRM technologies to disable the protection they offer.

◆ Auditing program binaries

- ◆ Open source software feels safer to run because it has been inspected and approved by thousands of impartial software engineers.
- ◆ RE is a viable (albeit limited) alternative for searching for security vulnerabilities when the source code of the program is not available.

Software Development-Reversing

◆ Achieving Interoperability with proprietary software

- ◆ Proprietary software libraries or OS APIs often have insufficient documentation.
- ◆ Solutions:
 - ◆ Try to get things to work via brute force.
 - ◆ Contact vendor for answers.
 - ◆ Use RE to figure out how the code works.

◆ Developing competing software

- ◆ Even if the competitor has unpatented technology it is not cost-effective to RE the entire product.
- ◆ The exception may be highly complex or unique designs/algorithms that are very difficult or costly to develop.
- ◆ There are legal issues to consider when RE competing software.

◆ Evaluating software quality and robustness

- ◆ It is possible to audit software to get an estimate of the general quality of the coding practices used in the program.
- ◆ Microsoft's Windows source code is not publicly available, and the source code is released to only a few for evaluation purposes.
- ◆ The rest have to either trust Microsoft or resort to RE.
- ◆ Access to source code is more desirable than binary reversing, however – why?

RE exercise

◆ General purpose registers:

- ◆ EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
- ◆ ESP usually for stack pointer
- ◆ EBP usually for base pointer

◆ `mov` – moves data between registers and memory from source to destination

◆ `mov` has variants – `movb` (1 byte), `movw` (2 bytes), `movl` (3 bytes)

◆ `$1` is the value 1

- ◆ E.g., `movl $1, %edx` means move 32-bit integer value 1 to the address in `EDX`

◆ `jmp` – go to the label specified.

◆ `jne` – go to the label if not equal.

◆ `imul` – multiply two, and save onto the second operand.

◆ `decl` – decrement by 1

◆ `Ret` - return

Note: this is based on a specific OS on X86 arch. The assembly code will differ based on the OS, CPU arch and the compiler used.

```
1      .file      "factorial.c"
2      .text
3      .globl factorial
4      .type      factorial, @function
5      factorial:
6          testl   %eax, %eax
7          jne     .L2
8          movl    $1, %edx
9          jmp     .L4
10     .L2:
11         movl    $1, %edx
12     .L5:
13         imull   %eax, %edx
14         decl    %eax
15         jne     .L5
16     .L4:
17         movl    %edx, %eax
18         ret
19     .size      factorial, .-factorial
20     .ident     "GCC: (GNU) 4.1.2 20060715 (prerelease) (Debian 4.1.1-9)"
21     .section   .note.GNU-stack,"",@progbits
```

RE exercise

factorial:

```
    if ((eax & eax) != 0){
        go to .L2;
    }
    else{
        edx = 1;
        go to .L4;
    }
.L2:
    edx = 1;
.L5:
    edx = eax * edx;
    eax = eax - 1;
    if(eax != 0){
        go to .L5;
    }
.L4:
    eax = edx;
    return;
```

```
1      .file      "factorial.c"
2      .text
3      .globl factorial
4      .type      factorial, @function
5      factorial:
6          testl    %eax, %eax
7          jne      .L2
8          movl     $1, %edx
9          jmp      .L4
10     .L2:
11         movl     $1, %edx
12     .L5:
13         imull    %eax, %edx
14         decl     %eax
15         jne      .L5
16     .L4:
17         movl     %edx, %eax
18         ret
19     .size      factorial, .-factorial
20     .ident     "GCC: (GNU) 4.1.2 20060715 (prerelease) (Debian 4.1.1-9)"
21     .section    .note.GNU-stack,"",@progbits
```

Reverse engineer this code!

Is RE legal?

- ◆ What social and economic impact does RE have on society?

Legal Issues:

Interoperability

- ◆ Exposing software interfaces facilitates the development of software that runs on top of the exposed platform.
 - ◆ Good for increasing the sales of the platform.
 - ◆ Bad if the competition offers a better product on the same platform.
- ◆ US court in legal case of Sega (original) versus Accolade (copied) ruled in favor of Accolade.
 - ◆ Accolade did not violate code copyright of Sega.
 - ◆ Competition in the market benefited customers.
 - ◆ Court essentially authorized RE for the purpose of interoperability.

Legal Issues:

Competition

- ◆ RE for interoperability has societal benefits, but when RE is used to develop competing products the situation is more complicated.
- ◆ Opponents of RE claim that reversing stifles innovation.
- ◆ Illegal (easy to prove): Directly stealing code.
- ◆ Illegal (hard to prove): Decompiling a program and recompiling it to generate a different binary with the same functionality.
- ◆ Legal: RE small parts of a product to gather information, not code. Then develop code independently.

Legal Issues:

Copyright Law

- ◆ A violation of copyright law is to directly copy protected code sequences from the competitor's product into your own product.
- ◆ Some have claimed that intermediate copies during RE decompilation violate copyright.
 - ◆ These claims did not hold in court (e.g., Sega versus Accolade) because intermediate copies are created when a program is installed from a CD onto a hard disk.
- ◆ If the final product does not contain anything that was directly copied from the original product, copying is considered fair use.

Legal Issues:

Trade Secrets and Patents

◆ **Patent:**

- ◆ Owner controls the invention for up to 20 years (or so, depending on the jurisdiction).
- ◆ Details of the patent are publicly disclosed, hence RE of a patented technology does not make sense.
- ◆ Invention becomes public after patent expires.
- ◆ Patent protects owner from a competitor who independently invents the same technology.

◆ **Trade secret:**

- ◆ Granted automatically if product is kept a secret and significant effort was put into its development.
- ◆ Protects against rogue employee selling secrets to the competition.
- ◆ Does not protect product from RE.

Legal Issues:

The Digital Millennium Copyright Act (DMCA)

- ◆ DMCA legally protects *copyright protection systems* from circumvention.
- ◆ DMCA is considered *anti-RE legislation* because such circumvention almost always involves reversing.
- ◆ DMCA only applies to copyright protection systems, so most RE applications are not affected by it.
- ◆ Two high-profile cases of DMCA violation:
 - ◆ Felten versus RIAA (audio recordings protection)
 - ◆ US versus Sklyarov (Adobe eBook file format)

Legal Issues:

The Digital Millennium Copyright Act (DMCA) – What is not allowed?

1. Circumvention of copyright protection systems.
 - E.g., using a *keygen* program.
 2. The development of circumvention technologies (DRM Digital Rights Management).
 - E.g., developing a *keygen* program.
- ◆ A *keygen* program generates a serial number on-the-fly for programs that require a serial number during installation.

Legal Issues:

The Digital Millennium Copyright Act (DMCA) - What is allowed?

- Interoperability
- Encryption research on encryption products
- Security testing
- Educational institutions and public libraries can violate DMCA in order to evaluate a copyrighted work prior to purchasing it.
- Government investigation
- Regulation
- Protection of privacy

Legal Issues:

License Agreement Considerations

- Other than DMCA, there are no laws that directly prohibit or restrict RE.
- DMCA only applies to DRM products.
- Vendors add anti-RE clauses to software license agreements.
- In the US it is not clear that these clauses are enforceable.
- In the EU decompilation for the purposes of interoperability is permissible under the “Directive of the Legal Protection of Computer Programs”.

The RE Process

◆ System-level reversing:

- ◆ Techniques that help determine the general structure of the program.
- ◆ Most of the information comes from the OS and, hence, OS monitoring utilities.

◆ Code-level reversing:

- ◆ Techniques provide detailed information on a selected code segment.
- ◆ Involves extracting design concepts and algorithms from binary code.

RE Tools

- ◆ System monitoring tools:
 - ◆ Network activity, file access, register access.
- ◆ Disassemblers:
 - ◆ Translate binary code to assembly language code.
- ◆ Debuggers:
 - ◆ Reversers use debuggers in disassembly mode to set breakpoints and step through a program's execution (with on-the-fly disassembly).
- ◆ Decompilers:
 - ◆ Attempt to produce high-level code (e.g., C) from an executable binary file. **A reverse compiler!**
 - ◆ Perfect decompilation impossible for most platforms.
 - ◆ Decompile a binary programs into readable source code.
 - ◆ Replace all binary code that could not be decompiled with assembly code.

Reverse Engineering Tool Categories

- ◊ Hex Editors
- ◊ Decompilers
- ◊ **Disassemblers/Debuggers**

Reverse Engineering Prevention Tools

- Code Obfuscators

Disassemblers/Debuggers

- Convert binary code into its assembly equivalent.
- Extract ASCII strings and used libraries.
- View memory, stack and CPU registers.
- Run the program (with breakpoints).
- Edit the assembly code at runtime.

Product	Dis-Assembly	Processor options	Debugger	String extraction	Disk Hex editor	Memory Hex editor	Memory Dumper	Library's used	Decryptor
<i>IDAPro</i>	x	x		x	x			x	
<i>OllyDbg</i>	x	x	x	x	x	x	x	x	
<i>W32Dasm</i>	x	x	x	x	x	x	x	x	
<i>BORG</i>	x	x					x		x

◆ Demo

◆ You should be able to use your Kali

Works best on AMD64 machine...

```
wget https://github.com/uwacyber/cits3006/raw/live/cits3006-labs/files/test_gdb
```

If you want to run on Apple Silicon...

Use the emulation on your machine (I shared the UTM file re_emulation_arm.utm)

Use this machine for doing this demo



Disassemblers/Debuggers: objdump

◆objdump -d [file]

```
ubuntu@lima-x86demo:~$ objdump -d test_gdb
```

```
000011cd <validate_password>:
```

11cd:	55	push	%ebp
11ce:	89 e5	mov	%esp,%ebp
11d0:	e8 16 01 00 00	call	12eb <__x86.get_pc_thunk.ax>
11d5:	05 f3 2d 00 00	add	\$0x2df3,%eax
11da:	8b 45 08	mov	0x8(%ebp),%eax
11dd:	0f b6 00	movzbl	(%eax),%eax
11e0:	3c 68	cmp	\$0x68,%al
11e2:	75 3b	jne	121f <validate_password+0x52>
11e4:	8b 45 08	mov	0x8(%ebp),%eax
11e7:	83 c0 01	add	\$0x1,%eax
11ea:	0f b6 00	movzbl	(%eax),%eax
11ed:	3c 65	cmp	\$0x65,%al
11ef:	75 2e	jne	121f <validate_password+0x52>
11f1:	8b 45 08	mov	0x8(%ebp),%eax
11f4:	83 c0 02	add	\$0x2,%eax
11f7:	0f b6 00	movzbl	(%eax),%eax
11fa:	3c 6c	cmp	\$0x6c,%al
11fc:	75 21	jne	121f <validate_password+0x52>
11fe:	8b 45 08	mov	0x8(%ebp),%eax
1201:	83 c0 03	add	\$0x3,%eax
1204:	0f b6 00	movzbl	(%eax),%eax
1207:	3c 6c	cmp	\$0x6c,%al
1209:	75 14	jne	121f <validate_password+0x52>
120b:	8b 45 08	mov	0x8(%ebp),%eax
120e:	83 c0 04	add	\$0x4,%eax
1211:	0f b6 00	movzbl	(%eax),%eax
1214:	3c 6f	cmp	\$0x6f,%al
1216:	75 07	jne	121f <validate_password+0x52>
1218:	b8 00 00 00 00	mov	\$0x0,%eax
121d:	eb 05	jmp	1224 <validate_password+0x57>
121f:	b8 ff ff ff ff	mov	\$0xffffffff,%eax
1224:	5d	pop	%ebp
1225:	c3	ret	

Disassemblers/Debuggers: strings

◆ strings [file]

```
ubuntu@lima-x86demo:~$ strings test_gdb
td0
/lib/ld-linux.so.2
_IO_stdin_used
puts
strlen
__libc_start_main
__cxa_finalize
printf
__isoc99_scanf
__stack_chk_fail
libc.so.6
GLIBC_2.7
GLIBC_2.1.3
GLIBC_2.4
GLIBC_2.0
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
<hu;
<eu.
<lu!
Enter password:
Success!
Failure!
;*2$"
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_ctors_aux
completed.0
```

Disassemblers/Debuggers: GDB

◆ You will do more in the labs

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test_gdb...
(No debugging symbols found in test_gdb)
(gdb) info func
All defined functions:

Non-debugging symbols:
0x00001000  _init
0x00001030  __libc_start_main@plt
0x00001040  printf@plt
0x00001050  __stack_chk_fail@plt
0x00001060  puts@plt
0x00001070  strlen@plt
0x00001080  __isoc99_scanf@plt
0x00001090  __cxa_finalize@plt
0x000010a0  _start
0x000010d0  __x86.get_pc_thunk.bx
0x000010e0  deregister_tm_clones
0x00001120  register_tm_clones
0x00001170  __do_global_ctors_aux
0x000011c0  frame_dummy
0x000011c9  __x86.get_pc_thunk.dx
0x000011cd  validate_password
0x00001226  main
0x000012eb  __x86.get_pc_thunk.ax
0x000012f0  __stack_chk_fail_local
0x00001304  _fini
(gdb) █
```

Disassemblers/Debuggers: GDB

Dump of assembler code for function main:

```
0x00001226 <+0>:    lea     0x4(%esp),%ecx
0x0000122a <+4>:    and     $0xffffffff0,%esp
0x0000122d <+7>:    push    -0x4(%ecx)
0x00001230 <+10>:   push    %ebp
0x00001231 <+11>:   mov     %esp,%ebp
0x00001233 <+13>:   push    %ebx
0x00001234 <+14>:   push    %ecx
0x00001235 <+15>:   sub     $0x20,%esp
0x00001238 <+18>:   call    0x10d0 <__x86.get_pc_thunk.bx>
0x0000123d <+23>:   add     $0x2d8b,%ebx
0x00001243 <+29>:   mov     %ecx,%eax
0x00001245 <+31>:   mov     0x4(%eax),%eax
0x00001248 <+34>:   mov     %eax,-0x1c(%ebp)
0x0000124b <+37>:   mov     %gs:0x14,%eax
0x00001251 <+43>:   mov     %eax,-0xc(%ebp)
0x00001254 <+46>:   xor     %eax,%eax
0x00001256 <+48>:   sub     $0xc,%esp
0x00001259 <+51>:   lea     -0x1fc0(%ebx),%eax
0x0000125f <+57>:   push    %eax
0x00001260 <+58>:   call    0x1040 <printf@plt>
0x00001265 <+63>:   add     $0x10,%esp
0x00001268 <+66>:   sub     $0x8,%esp
0x0000126b <+69>:   lea     -0x12(%ebp),%eax
0x0000126e <+72>:   push    %eax
0x0000126f <+73>:   lea     -0x1faf(%ebx),%eax
0x00001275 <+79>:   push    %eax
0x00001276 <+80>:   call    0x1080 <__isoc99_scanf@plt>
0x0000127b <+85>:   add     $0x10,%esp
0x0000127e <+88>:   sub     $0xc,%esp
0x00001281 <+91>:   lea     -0x12(%ebp),%eax
0x00001284 <+94>:   push    %eax
0x00001285 <+95>:   call    0x1070 <strlen@plt>
0x0000128a <+100>:  add     $0x10,%esp
0x0000128d <+103>:  cmp     $0x5,%eax
0x00001290 <+106>:  jne     0x12b9 <main+147>
0x00001292 <+108>:  sub     $0xc,%esp
```

--Type <RET> for more, q to quit, c to continue without paging--

Disassemblers/Debuggers: GDB

```
(gdb) disas validate_password
```

```
Dump of assembler code for function validate_password:
```

```
0x000011cd <+0>:      push    %ebp
0x000011ce <+1>:      mov     %esp,%ebp
0x000011d0 <+3>:      call   0x12eb <__x86.get_pc_thunk.ax>
0x000011d5 <+8>:      add     $0x2df3,%eax
0x000011da <+13>:     mov     0x8(%ebp),%eax
0x000011dd <+16>:     movzbl (%eax),%eax
0x000011e0 <+19>:     cmp     $0x68,%al
0x000011e2 <+21>:     jne     0x121f <validate_password+82>
0x000011e4 <+23>:     mov     0x8(%ebp),%eax
0x000011e7 <+26>:     add     $0x1,%eax
0x000011ea <+29>:     movzbl (%eax),%eax
0x000011ed <+32>:     cmp     $0x65,%al
0x000011ef <+34>:     jne     0x121f <validate_password+82>
0x000011f1 <+36>:     mov     0x8(%ebp),%eax
0x000011f4 <+39>:     add     $0x2,%eax
0x000011f7 <+42>:     movzbl (%eax),%eax
0x000011fa <+45>:     cmp     $0x6c,%al
0x000011fc <+47>:     jne     0x121f <validate_password+82>
0x000011fe <+49>:     mov     0x8(%ebp),%eax
0x00001201 <+52>:     add     $0x3,%eax
0x00001204 <+55>:     movzbl (%eax),%eax
0x00001207 <+58>:     cmp     $0x6c,%al
0x00001209 <+60>:     jne     0x121f <validate_password+82>
0x0000120b <+62>:     mov     0x8(%ebp),%eax
0x0000120e <+65>:     add     $0x4,%eax
0x00001211 <+68>:     movzbl (%eax),%eax
0x00001214 <+71>:     cmp     $0x6f,%al
0x00001216 <+73>:     jne     0x121f <validate_password+82>
0x00001218 <+75>:     mov     $0x0,%eax
0x0000121d <+80>:     jmp     0x1224 <validate_password+87>
0x0000121f <+82>:     mov     $0xffffffff,%eax
0x00001224 <+87>:     pop     %ebp
0x00001225 <+88>:     ret
```

```
End of assembler dump.
```

```
(gdb) █
```

Disassemblers/Debuggers: GDB

```
gef> r
Starting program: /home/ubuntu/test_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password: aaaaa

Breakpoint 1, 0x565561d0 in validate_password ()
[ Legend: Modified register | Code | Heap | Stack | String ]

$eax : 0xffffd3c6 → "aaaaa"
$ebx : 0x56558fc8 → 0x00003ed0
$ecx : 0x6
$edx : 0x5
$esp : 0xffffd398 → 0xffffd3d8 → 0x00000000
$ebp : 0xffffd398 → 0xffffd3d8 → 0x00000000
$esi : 0xffffd4ac → 0xffffd606 → "SHELL=/bin/bash"
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x565561d0 → <validate_password+0003> call 0x565562eb <__x86.get_pc_thunk.ax>
$eflags: [zero carry parity ADJUST SIGN trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
```

registers

```
0x565561cd <validate_password+0000> push    %ebp
0x565561ce <validate_password+0001> mov     %esp, %ebp
➔ 0x565561d0 <validate_password+0003> call   0x565562eb <__x86.get_pc_thunk.ax>
↳ 0x565562eb <__x86.get_pc_thunk.ax+0000> mov     (%esp), %eax
0x565562ee <__x86.get_pc_thunk.ax+0003> ret
0x565562ef <__x86.get_pc_thunk.ax+0004> nop
```

code:x86:32

```
gef> □
```

```
gef> si
0x565561e0 in validate_password ()
2: x/cb $pc+1 0x565561e1 <validate_password+20>:      104 'h'
3: /c $al = 97 'a'
[ Legend: Modified register | Code | Heap | Stack | String ]
```

registers

```
$eax : 0x61
$ebx : 0x56558fc8 → 0x00003ed0
$ecx : 0x26
$edx : 0x5
$esp : 0xffffd378 → 0xffffd3b8 → 0x00000000
$ebp : 0xffffd378 → 0xffffd3b8 → 0x00000000
$esi : 0xffffd48c → 0xffffd5f9 → "SHELL=/bin/bash"
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x565561e0 → <validate_password+0013> cmp $0x68, %al
$eflags: [zero carry parity adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
```

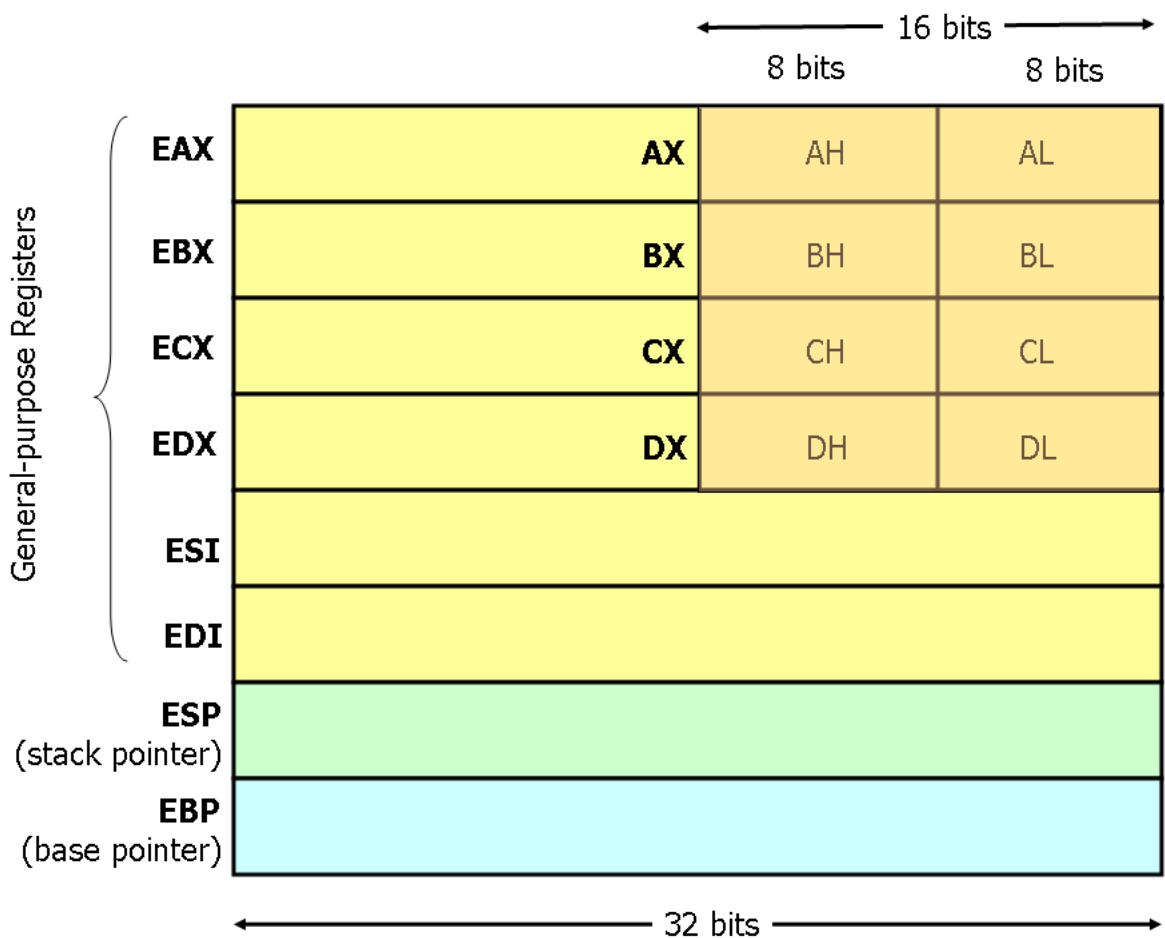
code:x86:32

```
0x565561da <validate_password+000d> mov     0x8(%ebp), %eax
0x565561dd <validate_password+0010> movzbl (%eax), %eax
➔ 0x565561e0 <validate_password+0013> cmp     $0x68, %al
0x565561e2 <validate_password+0015> jne     0x5655621f <validate_password+82>
0x565561e4 <validate_password+0017> mov     0x8(%ebp), %eax
```

```
gef> ▮
```

```
echo -e "\x68"
```

Disassemblers/Debuggers: GDB



◆demo

- You need a Windows VM and install x64dbg
 - tested on Windows 7 VM
 - ARM-based, you can use the provided Windows 7 VM that comes preconfigured

wget <https://github.com/uwacyber/cits3006/raw/live/cits3006-labs/files/program.exe> -O program.exe



Disassemblers/Debuggers: x64dbg

- ◆ Executes program in a controlled environment.
- ◆ Allows the flow of the program to be controlled.
- ◆ Uses a convenient layout showing hexadecimal, assembly, CPU registers and stack.
- ◆ Allows the program to be dumped from the memory onto the hard-disk.
- ◆ Highlights recently changed values in memory/stack/CPU registers.

The screenshot displays the x64dbg debugger interface for a program named 'program.exe' (PID: 2104, Module: ntdll.dll, Thread: Main Thread 2284). The interface is divided into several panes:

- Top Panel:** Shows the CPU register values. EAX is 00000000, EBX is 00000000, ECX is 57780000, EDX is 001AE348, EBP is 004CF934, ESP is 004CF908, ESI is FFFFFFFF, and EDI is 00000000. The EIP (Instruction Pointer) is 77930F3C, pointing to the instruction 'mov dword ptr ss:[ebp-4], esi'.
- Assembly Pane:** Displays the disassembled code. The current instruction is 'mov dword ptr ss:[ebp-4], esi' at address 77930F3C. Other visible instructions include 'jmp ntdll.77930F4F', 'xor eax, eax', 'inc eax', 'ret', 'mov esp, dword ptr ss:[ebp-18]', 'mov dword ptr ss:[ebp-4], FFFFFFFF', 'call ntdll.778BDEF9', 'ret', 'nop', 'mov edi, edi', 'push ebp', 'mov ebp, esp', 'sub esp, 10', 'cmp byte ptr ds:[7FFE02EC], 0', 'je ntdll.77930F7C', and 'mov eax, dword ptr ss:[ebp+C]'. The status bar indicates 'System breakpoint reached!'.
- Memory Map Pane:** Shows the memory map with various segments and their addresses.
- Call Stack Pane:** Displays the call stack with the current function 'ntdll.77930F3C' at the top.
- Script Pane:** Shows the script being executed, which is currently empty.
- Symbols Pane:** Lists the symbols loaded by the debugger, including 'ntdll.77930F3C'.
- Source Pane:** Shows the source code of the program, which is currently empty.
- Registers Pane:** Displays the values of the CPU registers, including EAX, EBX, ECX, EDX, EBP, ESP, ESI, and EDI.
- Memory Dump Pane:** Shows a dump of memory at address 004CF930, displaying hexadecimal and ASCII values. The dump includes the instruction 'return to ntdll.RtlUlong'.

The Command window at the bottom shows the command 'mov eax, ebx' and the status 'Time Wasted Debugging: 0:00:13:24'.

Disassemblers/Debuggers: x64dbg

program.exe - PID: 2104 - Module: ntdll.dll - T

File View Debug Tracing Plugins Favourite

CPU Log Notes Breakpoints

EIP 77930F3C 8975 FC

77930F3F EB 0E

77930F41 33C0

77930F43 40

77930F44 C3

77930F45 8865 E8

77930F48 C745 FC FEFF

77930F4F E8 A5CFF8FF

77930F54 C3

77930F55 90

77930F56 90

77930F57 90

77930F58 90

77930F59 90

77930F5A 88FF

77930F5C 55

77930F5D 88EC

77930F5F 83EC 10

77930F62 803D EC02FE7

77930F69 74 11

77930F6B 8845 0C

dwrd ptr ss:[ebp-04]=[004CF930]=0

esi=FFFFFFFF

.text:77930F3C ntdll.dll:\$A0F3C #9133

Dump 1 Dump 2 Dump 3

Address Hex

778A0000 8B 44 24 04 CC C2 04 00 CC 90 C3 90 CC C3 90 90 .DS.IA..I.A.IA..

778A0010 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .L.S.A..t.e..

778A0020 8B 4C 24 04 F6 41 04 06 74 05 E8 81 1D 01 00 88 .L.S.A..t.e..

778A0030 01 00 00 00 C2 10 00 00 8D 84 24 DC 02 00 00 64 .A..A..S.U..

778A0040 8B 0D 00 00 00 00 BA 20 00 8A 77 89 08 89 50 04 .L.S.A..t.e..

778A0050 64 A3 00 00 00 00 58 8D 7C 24 0C FF D0 8B 8F CC .L.S.A..t.e..

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:00:12:26

Graph Help on mnemonic Ctrl+F1 Show mnemonic brief Ctrl+Shift+F1 Highlighting mode H Edit columns... Label Comment Toggle Bookmark Ctrl+D Trace coverage Analysis Download Symbols for This Module Assemble Space Patches Ctrl+P Set EIP Here Ctrl+* Create New Thread Here Go to Search for Find references to Current Region Current Module All User Modules All System Modules All Modules

EAX 00000000 EBX 00000000 ECX 57780000 EDX 001AE348 EBP 004CF934 ESP 004CF908 ESI FFFFFFFF EDI 00000000

EIP 77930F3C ntdll.77930F3C

EFLAGS 00000246 ZF 1 PF 1 AF 0 OF 0 SF 0 DF 0

Default (stdcall) 5 Unlocked

1: [esp+4] 00000000 00000000

2: [esp+8] 00000000 00000000

3: [esp+C] 7EFDE000 7EFDE000

4: [esp+10] 00000000 00000000

5: [esp+14] 004CF908 004CF908

program.exe - PID: 2104 - Module: program.exe - Thread: Main Thread 2284 - x32dbg

File View Debug Tracing Plugins Favourite Options Help May 27 2026 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads

012F16A3 C70424 5CA02F01 mov dword ptr ss:[esp],program.12FA12FA05C:"Enter the pas

012F16A4 E8 11FFFFFF call program.12F15C0

012F16A5 C70424 00000000 mov dword ptr ss:[esp],0

012F16A6 A1 48902F01 mov eax,dword ptr ds:[12F9048]

012F16A7 FF D0 call eax

012F16A8 894424 08 mov dword ptr ss:[esp+8],eax

012F16A9 C74424 04 14000000 mov dword ptr ss:[esp+4],14

012F16AA 8D4424 1C lea eax,dword ptr ss:[esp+1C]

012F16AB 890424 mov dword ptr ss:[esp],eax

012F16AC E8 0B6A0000 call <JMP.&gets>

012F16AD 8D4424 1C lea eax,dword ptr ss:[esp+1C]

012F16AE 890424 mov dword ptr ss:[esp],eax

012F16AF E8 30FFFFFF call program.12F1611

012F16B0 85C0 test eax,eax

012F16B1 74 0E je program.12F16F3

012F16B2 C70424 71A02F01 mov dword ptr ss:[esp],program.12FA12FA071:"LOCKED"

012F16B3 E8 89FFFFFF call program.12F167A

012F16B4 EB 18 jmp program.12F1708

012F16B5 C70424 78A02F01 mov dword ptr ss:[esp],program.12FA12FA078:"The program f

012F16B6 E8 C1FFFFFF call program.12F15C0

012F16B7 C70424 92A02F01 mov dword ptr ss:[esp],program.12FA12FA092:"err"

dwrd ptr ss:[esp]=[004CF908]=77DB1F34

012FA05C "Enter the password: "

.text:012F16A3 program.exe:\$16A3 #AA3

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 [x] Locals 004CF908 77DB1F34

Address Hex

778A0000 8B 44 24 04 CC C2 04 00 CC 90 C3 90 CC C3 90 90 .DS.IA..I.A.IA..

778A0010 90 90 90 90 90 90 90 90 90 90 90 90 90 90 .L.S.A..t.e..

778A0020 8B 4C 24 04 F6 41 04 06 74 05 E8 81 1D 01 00 88 .L.S.A..t.e..

778A0030 01 00 00 00 C2 10 00 00 8D 84 24 DC 02 00 00 64 .A..A..S.U..

778A0040 8B 0D 00 00 00 00 BA 20 00 8A 77 89 08 89 50 04 .L.S.A..t.e..

778A0050 64 A3 00 00 00 00 58 8D 7C 24 0C FF D0 8B 8F CC .L.S.A..t.e..

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused 3822 string(s) in 8907ms Time Wasted Debugging: 0:00:15:23

EAX 00000000 EBX 00000000 ECX 57780000 EDX 001AE348 EBP 004CF934 ESP 004CF908 ESI FFFFFFFF EDI 00000000

EIP 77930F3C ntdll.77930F3C

EFLAGS 00000246 ZF 1 PF 1 AF 0 OF 0 SF 0 DF 0

Default (stdcall) 5 Unlocked

1: [esp+4] 00000000 00000000

2: [esp+8] 00000000 00000000

3: [esp+C] 7EFDE000 7EFDE000

4: [esp+10] 00000000 00000000

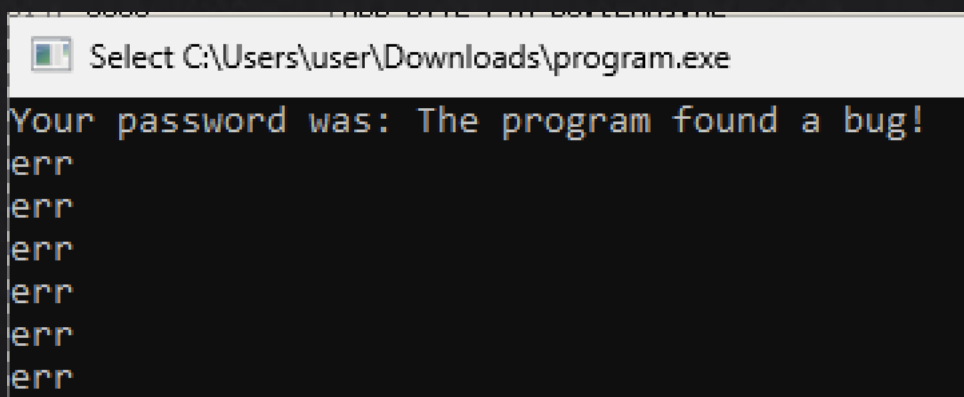
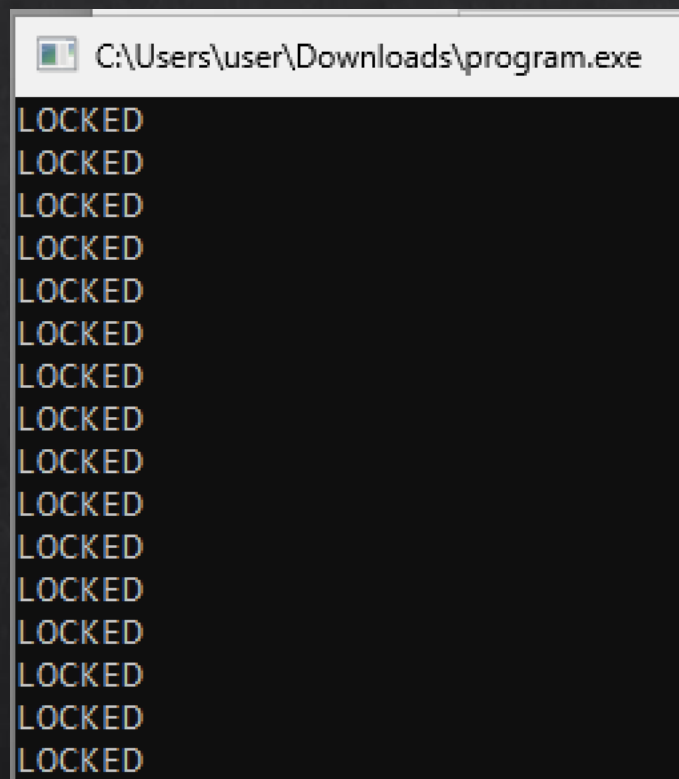
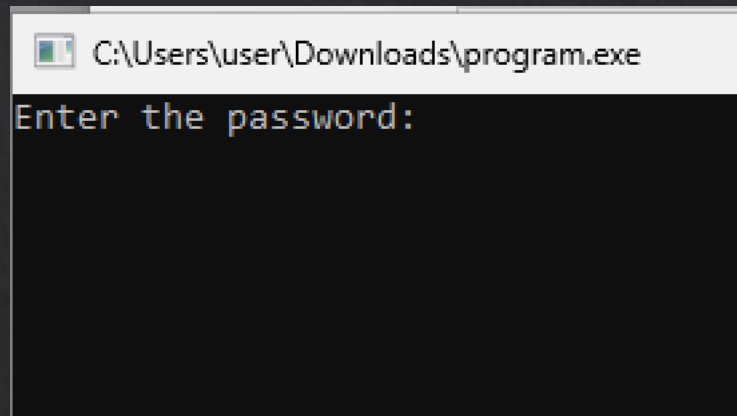
5: [esp+14] 004CF908 004CF908

ntdll.winsqmSetIfMaxDWORD+35

Pointer to SEH_Record[1]

ntdll.winsqmSetIfMaxDWORD+35

Disassemblers/Debuggers: x64dbg



Disassemblers/Debuggers: x64dbg

program.exe - PID: 2620 - Module: program.exe - Thread: Main Thread 3036 - x32dbg

File View Debug Tracing Plugins Favourites Options Help May 27 2026 (TitanEngine)

Check for Updates

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads

EIP 01231650 E8 EB6A0000 call <JMP.&strncmp> eax:"alexddit"

01231655 85C0 test eax, eax

01231657 75 05 jne program.123165E

01231659 E8 9EFFFFFF call program.12315FC

0123165E C74424 08 09000000 mov dword ptr ss:[esp+8], 9

01231666 8B45 08 mov eax, dword ptr ss:[ebp+8]

01231669 894424 04 mov dword ptr ss:[esp+4], eax

0123166D 8D45 E4 lea eax, dword ptr ss:[ebp-1C]

01231670 890424 mov dword ptr ss:[esp], eax

01231673 E8 C86A0000 call <JMP.&strncmp> [esp]:"alexddit"

01231678 C9 leave

01231679 C3 ret

0123167A 55 push ebp

0123167B 89E5 mov ebp, esp

0123167D 83EC 18 sub esp, 18

01231680 8B45 08 mov eax, dword ptr ss:[ebp+8]

01231683 894424 04 mov dword ptr ss:[esp+4], eax

01231687 C70424 58A02301 mov dword ptr ss:[esp], program.123A

0123168E E8 2DFFFFFF call program.12315C0

01231693 EB EB jmp program.1231680

<JMP.&strncmp>

.text:01231650 program.exe:\$1650 #A50

Address Hex

778A0000 8B 44 24 04 CC C2 04 00 CC 90 C3 90 CC C3 90 90 90

778A0010 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90

778A0020 8B 4C 24 04 F6 41 04 06 74 05 E8 81 1D 01 00 88

778A0030 01 00 00 00 C2 10 00 90 8D 84 24 DC 02 00 00 64

778A0040 8B 00 00 00 00 00 BA 20 00 8A 77 89 08 89 50 04

778A0050 64 A3 00 00 00 00 58 8D 7C 24 0C FF D0 8B 8F CC

ASCII

..S..I..I..A..I..A..

.....

..L..S..O..A..t..e..

.....A.....\$U...d

.....W...P..

df....X..I\$.yD..I

003CFA90 003CFA9C "alexddit"

003CFA98 003CFAEC "fdafdasfs\n"

003CFA9C 00000009

003CFA98 007D0F5C msvcrt.771A46C9

003CFAA0 0000000A

003CFAA4 77222900 msvcrt._iob

003CFAA8 003CFAEC "fdafdasfs\n"

003CFAAC 78656C61

003CFAB0 69646964

EFLAGS 00000304

ZF 0 PF 1 AF 0

OF 0 SF 0 DF 0

Default (stdcall) 5 Unlocked

1: [esp] 003CFAAC 003CFAAC "alexddit"

2: [esp+4] 003CFAEC 003CFAEC "fdafdasfs\n"

3: [esp+8] 00000009 00000009

4: [esp+C] 007D0F5C 007D0F5C

5: [esp+10] 0000000A 0000000A

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Reverse Engineering Tool Categories

- ◊ Hex Editors
- ◊ Decompilers
- ◊ Disassemblers/Debuggers

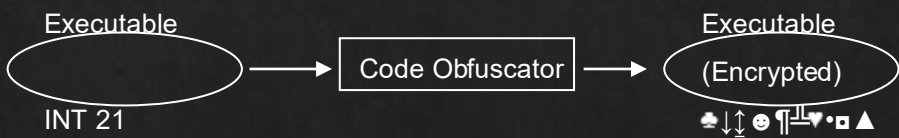
Reverse Engineering Prevention Tools

- **Code Obfuscators**

Code Obfuscators

- ◆ Encrypts the code of a program so you cannot view it in assembly.
- ◆ Disallows the program to be run if it detects a known disassembler or debugger running.

Obfuscators	Obfuscation	Anti-debugging techniques	GUI
<i>Y0da's Cryptor</i>	x	x	x
<i>NFO</i>	x	x	



Disassemblers/Debuggers: AI



program.exe
File

reverse engineer this file for me, which is part of a learning exercise.

Worked for 15s >

I reverse-engineered the uploaded executable statically. It's a **32-bit Windows PE console program**, apparently compiled with MinGW/GCC, and it's a deliberately simple password-checking exercise.

Key finding: the password

The password embedded directly in `_check_password()` is:

`alexdidit`

The relevant assembly constructs the string byte-by-byte:

```
61 6c 65 78 64 69 64 69 74
a l e x d i d i t
```



So the intended password is `alexdidit`.

Disassemblers/Debuggers: AI

◆ Now let's try this binary

```
wget https://github.com/uwacyber/cits3006/raw/live/cits3006-labs/files/ai_re_demo
```



Resources

◆ Reverse Engineering

- ◆ <https://0xinfection.github.io/reversing/>

◆ Decompiler tools

- ◆ <https://en.kali.tools/all/?category=decompiler>

References

- ◆ Materials adopted from
 - Spiros Mancoridis, Drexel University